Lesson 8

Performance Optimization, useMemo, useCallback & API Sort/Search

Code the Dream — React

Today's Session

EXPLORE

- Why components re-render — and why it matters
- `useMemo`: caching computed values
- `useCallback`: caching function references
- `React.memo`: preventing child re-renders
- Local vs. API-based sort and search
- Caching, throttling, and pagination

APPLY

- Observe a performance-heavy Book Dashboard with render counters
- Stabilize two event handlers with `useCallback`
- Optimize sorting and statistics with `useMemo`
- Measure the improvement with timing benchmarks

> By the end: identify unnecessary re-renders, apply the right memoization hook, and decide when to handle data locally vs. on the server.

EXPLORE

Why React Re-renders — And Why It Matters

Every time a component re-renders, React:

- **Redefines every function** in the component body — creating a new reference
- **Recalculates every expression** — including expensive computations
- **Re-renders all child components** by default — even if their props are the same

This is usually fast. But it becomes expensive when:

- A calculation is heavy (filtering 10,000 items, computing totals)
- A function is passed as a prop to many child components
- A child does a lot of work that doesn't need to repeat

> Two hooks solve this: `useMemo` caches a computed **value**. `useCallback` caches a **function reference**.

EXPLORE

`useMemo` — Caching Computed Values

`useMemo` runs a calculation only when its dependencies change — and reuses the cached result otherwise.

```
const filtered = useMemo(() => {
  return items.filter(item =>
    item.toLowerCase().includes(query.toLowerCase())
  );
}, [items, query]); // only re-filters when items or query changes
```

Key points:

- Returns the **result** of the computation (a value, array, object…)
- The function inside cannot take arguments — it reads from the surrounding scope
- Use for: expensive derived data, sorted/filtered lists, computed totals

> Without `useMemo`, this filter runs on every render — even when the user scrolls, hovers, or changes unrelated state.

EXPLORE

`useCallback` — Caching Function References

`useCallback` returns the same function reference between renders, as long as its dependencies haven't changed.

```
const handleIncrement = useCallback(() => {
  setCount(c => c + 1);
}, []); // empty deps → same reference every render
```

Key points:

- Returns the **function itself** — not its result
- Every render without `useCallback` creates a new function object — a new reference
- Use for: callbacks passed to memoized child components (`React.memo`)
- Use the setter function pattern (`c => c + 1`) inside `useCallback` to avoid stale closures

> If a child component receives a new function reference on every render, it re-renders — even if nothing actually changed. `useCallback` fixes this.

EXPLORE

`useMemo` vs. `useCallback`

	`useMemo`	`useCallback`
Returns	The **result** of a computation	The **function** itself
Purpose	Cache derived data	Stable function reference for child props
Syntax	`useMemo(() => compute(), [deps])`	`useCallback(() => {}, [deps])`
Use when	Calculation is expensive	Child component is memoized with `React.memo`
Common example	Filtered/sorted list	Event handler passed to a list of items

> Think of it this way: `useMemo` = save the answer. `useCallback` = save the question.

EXPLORE

`React.memo` + `useCallback` — The Full Picture

`React.memo` wraps a component and skips re-rendering it when its props haven't changed (shallow comparison).

```
// Child — memoized; skips re-render if props are the same
const ProductCard = memo(function ProductCard({ name, onAdd }) {
  return <button onClick={() => onAdd(name)}>{name}</button>;
});

// Parent — useCallback keeps onAdd stable across renders
const handleAdd = useCallback((name) => {
  setCart(prev => [...prev, name]);
}, []);
```

Without `useCallback`, `handleAdd` is a new function reference every render → `memo`'s shallow comparison fails → `ProductCard` re-renders anyway. The two must be used together.

Check for Understanding

A parent has both an expensive filtered list and an event handler it passes to a memoized child. Which statement is correct?

A) Use `useMemo` for the filter and `useCallback` for the handler — both are needed

B) Use `useMemo` for both — it can memoize functions too

C) Use `useCallback` for both — the two hooks do the same thing

D) Neither is needed — React is fast enough for most cases

EXPLORE

When NOT to Optimize

`useMemo`, `useCallback`, and `React.memo` add complexity and memory overhead. They can actually slow things down if misapplied.

Skip memoization when:

- The component is simple and renders quickly
- The function or value isn't passed to memoized children
- The dependency array changes on every render anyway (the cache is never reused)
- You haven't confirmed there's a real performance problem

The right order:

1. Write clear, correct code first
2. Measure — use React DevTools Profiler or render counters
3. Optimize only the bottleneck you confirmed

EXPLORE

Local vs. API-Based Sort and Search

Local — download everything, manipulate in the browser

```
const sorted = useMemo(() =>
  [...users].sort((a, b) =>
    a.name.localeCompare(b.name))
, [users]);
```

✅ Instant feedback, works offline
❌ Large download, stale data risk

Use when: dataset < ~1,000 items

API-based — let the server filter and sort

```
const resp = await fetch(
  `/api/users?sort=name&page=${page}`
);
```

✅ Handles massive datasets, always fresh
❌ Network delay on every interaction

Use when: dataset > 10,000 items, data changes frequently

EXPLORE

Caching API Results

If a user searches for "React" twice, there's no reason to hit the API a second time. Store results in a lookup object keyed by the query.

```
const [cache, setCache] = useState({});

async function handleSearch(query) {
  if (cache[query]) {
    setResults(cache[query]); // use cached results
    return;
  }
  const data = await fetchResults(query);
  setResults(data);
  setCache(prev => ({ ...prev, [query]: data })); // save to cache
}
```

> This is especially important when an API has a limited daily request quota. Caching prevents wasting quota on repeated searches for the same term.

EXPLORE

Throttling & Pagination

Throttling — limit how fast requests can fire

```
function handlePageForward() {
  setIsDisabled(true);
  setPage(p => p + 1);
  setTimeout(
    () => setIsDisabled(false),
    1000 // match API rate limit
  );
}
```

Disabling the button prevents rapid-fire requests that could exhaust an API's rate limit.

Pagination — load one page at a time

```
const [page, setPage] = useState(1);

useEffect(() => {
  fetchUsers(page);
}, [page]);
// changing page triggers a new fetch
```

Page state drives the fetch — incrementing or decrementing page fetches the next or previous set of results.

Now: Apply

Let's optimize a sluggish Book Dashboard — step by step.

APPLY

This Week's Exercise — Book Library Dashboard

Open `src/exercises/lesson-08/studentWork.jsx`.

The app has 100 books with search, filtering, sorting, and a favorites system. **It works — but it's slow.** Every interaction causes unnecessary re-renders and expensive recalculations.

Built-in measurement tools:

- **Render counters** — visible in the corner of each component, increment on every re-render
- **Timing benchmarks** — `BookStats` displays calculation time in microseconds

APPLY

Your job — four TODOs:

TODO	Location	Fix
#1	`StudentWork`	Wrap `handleSearch` in `useCallback`
#2	`StudentWork`	Wrap `handleToggleFavorite` in `useCallback`
#3	`BookList`	Wrap sort logic in `useMemo`
#4	`BookStats`	Wrap `calculateStats()` in `useMemo`

APPLY

Step 1 — Stabilize `handleSearch` with `useCallback`

Every keystroke re-renders the parent, creating a new `handleSearch` reference — which triggers all `BookCard` components to re-render.

Before

```
function handleSearch(e) {
  setSearchTerm(e.target.value);
}
```

After

```
const handleSearch = useCallback((e) => {
  setSearchTerm(e.target.value);
}, []); // no dependencies — setter is always stable
```

Test: Type in the search box and verify that `BookCard` render counters no longer increment.

APPLY

Step 2 — Stabilize `handleToggleFavorite` with `useCallback`

Same problem: clicking any favorite recreates the function and re-renders all `BookCard` components.

```
const handleToggleFavorite = useCallback((bookId) => {
  setFavorites(prev =>
    prev.includes(bookId)
      ? prev.filter(id => id !== bookId)
      : [...prev, bookId]
  );
}, []); // setter pattern avoids reading favorites directly
```

The functional update (`prev => ...`) reads the latest state without adding `favorites` to the dependency array. This keeps the reference stable.

Test: Click "Add to Favorites" — only the clicked card should re-render, not all cards.

APPLY

Step 3 — Optimize Sorting with `useMemo` in `BookList`

The sort operation runs on every render — even when the book list and sort order haven't changed.

Before

```
const sortedBooks = books.toSorted((a, b) => {
  // ...sort logic based on sortBy
});
```

After

```
const sortedBooks = useMemo(() => {
  return books.toSorted((a, b) => {
    // ...sort logic based on sortBy
  });
}, [books, sortBy]); // only re-sort when books or sortBy changes
```

Test: Change the sort option — sorting should run. Type in the search box — sorting should not run again.

APPLY

Step 4 — Optimize Statistics with `useMemo` in `BookStats`

`calculateStats()` contains an artificial 20,000-iteration loop to simulate expensive computation. Without memoization, it runs on every render.

Before

```
const stats = calculateStats(books);
// runs every render — watch the timing display
```

After

```
const stats = useMemo(() => {
  return calculateStats(books);
}, [books]); // only recalculate when books changes
```

Test: After applying, type in the search box. The `BookStats` timing display should drop to `~0.00ms` for renders where `books` hasn't changed — the cache is being reused.

Wrap-Up

This week you learned:

- Every render redefines functions and recalculates values — `useMemo` and `useCallback` prevent unnecessary work
- `useMemo` caches a computed **value**; `useCallback` caches a **function reference**
- `React.memo` skips child re-renders — but only works when function props are stabilized with `useCallback`
- Measure first, optimize only the confirmed bottleneck
- For large datasets: API-based sort/search + caching + pagination beats local manipulation

> **Before next session:** Complete the weekly assignment — add pagination to your todo list, implement API-based search, and apply `useCallback` to stabilize your event handlers. Submit your PR link in CTD Learns. Post questions in **#[class-name]-discussion** on Slack.